UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas
UNIDAD DE POSGRADO

Apunte de Clase — LeNet-5: la primera CNN moderna

En esta clase

LeNet-5 (LeCun et al., 1998) es la arquitectura que introdujo las ideas centrales de las redes convolucionales modernas: convolución, pooling y capas densas encadenadas para reconocimiento de imágenes. Es el punto de partida natural antes de AlexNet, VGG, GoogLeNet y ResNet.

I. Contexto histórico

LeNet-5 fue propuesta por Yann LeCun y colaboradores en 1998 en el artículo "Gradient-Based Learning Applied to Document Recognition", con el objetivo de reconocer dígitos escritos a mano para el procesamiento automático de cheques bancarios en Estados Unidos.

Aunque hoy parece una red pequeña (~60,000 parámetros, frente a los millones o miles de millones de las redes actuales), en 1998 fue un hito: demostró que una red neuronal entrenada de punta a punta con backpropagation podía superar a los sistemas de reconocimiento de patrones hechos a mano que dominaban la industria en ese momento.

II. ¿Por qué seguimos estudiando LeNet-5?

- Introduce el patrón arquitectónico que toda CNN moderna sigue: bloques de (convolución → activación → pooling) repetidos, seguidos de capas totalmente conectadas.
- Es lo suficientemente pequeña para entrenarse en segundos/minutos incluso sin GPU, ideal para entender cada paso del forward y backward pass.
- Sienta las bases conceptuales de AlexNet (2012), que es esencialmente una LeNet mucho más profunda y grande entrenada con GPUs.

III. Arquitectura de LeNet-5

La arquitectura original recibe imágenes de 32×32 píxeles en escala de grises y las procesa a través de 7 capas con parámetros entrenables:

Capa	Tipo	Entrada	Salida	Detalle
C1	Convolución	32×32×1	28×28×6	6 filtros de 5×5, stride 1
S2	Pooling (avg.)	28×28×6	14×14×6	ventana 2×2, stride 2
C3	Convolución	14×14×6	10×10×16	16 filtros de 5×5, stride 1
S4	Pooling (avg.)	10×10×16	5×5×16	ventana 2×2, stride 2
C5	Convolución (equiv. a FC)	5×5×16	1×1×120	120 filtros de 5×5
F6	Totalmente conectada	120	84	activación tanh

Capa	Tipo	Entrada	Salida	Detalle
Salida	Totalmente conectada	84	10	una neurona por dígito (0-9)

IV. Aritmética de la convolución

El tamaño espacial de la salida de una capa convolucional (o de pooling) se calcula con:

$$O = piso[\ (W - K + 2P) / S\ ] + 1$$

donde W es el tamaño de entrada, K el tamaño del kernel, P el padding y S el stride. Por ejemplo, para C1: W=32, K=5, P=0, S=1 → O = (32−5)/1 + 1 = 28, que coincide con la salida 28×28 de la tabla.

V. Número de parámetros por capa

Para una capa convolucional, el número de parámetros entrenables es (K×K×C_in + 1) × C_out (el +1 es el bias de cada filtro). Para una capa totalmente conectada es (n_in + 1) × n_out.

Capa	Cálculo	Parámetros
C1	(5×5×1 + 1) × 6	156
C3	(5×5×6 + 1) × 16	2,416
C5	(5×5×16 + 1) × 120	48,120
F6	(120 + 1) × 84	10,164
Salida	(84 + 1) × 10	850

Total ≈ 61,700 parámetros. (La LeNet-5 original de 1998 usaba conexiones dispersas entre S2 y C3 para reducir este número; casi todas las implementaciones modernas, incluida la de este apunte, usan conexión completa por simplicidad.)

VI. LeNet-5 original vs. implementación moderna

Aspecto	LeNet-5 original (1998)	Implementación moderna típica
Activación	tanh o sigmoide	ReLU
Pooling	Average pooling entrenable (con coeficientes)	Max pooling (sin parámetros)
Conexión S2→C3	Dispersa (cada filtro ve solo algunos mapas de entrada)	Completa (cada filtro ve todos los mapas de entrada)
Capa de salida	RBF (Radial Basis Function)	Softmax / CrossEntropyLoss

El código de la siguiente sección usa las convenciones modernas (ReLU y max pooling), manteniendo el resto de la arquitectura fiel al diseño original.

VII. Implementación en PyTorch (Google Colab)

El siguiente código define LeNet-5 y la entrena sobre MNIST (28×28, la red original esperaba 32×32; se soluciona con padding=2 en la primera convolución). Está listo para copiar y ejecutar en Google Colab; el notebook adjunto (LeNet5_MNIST.ipynb) contiene el mismo código ya organizado en celdas.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_data = datasets.MNIST("./data", train=True, download=True,
transform=transform)
test_data  = datasets.MNIST("./data", train=False, download=True,
transform=transform)

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader  = DataLoader(test_data, batch_size=1000, shuffle=False)

class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # padding=2 simula la entrada 32x32 original a partir de MNIST (28x28)
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, padding=2)   # 28x28 -> 28x28
        self.pool1 = nn.MaxPool2d(2)                             # 28x28 -> 14x14
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)             # 14x14 -> 10x10
        self.pool2 = nn.MaxPool2d(2)                             # 10x10 -> 5x5
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, num_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool1(self.relu(self.conv1(x)))
        x = self.pool2(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = LeNet5().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(5):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
```

```
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}")

model.eval()
correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images).argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
print(f"Test Accuracy: {100*correct/total:.2f}%")
print(f"Parámetros entrenables: {sum(p.numel() for p in model.parameters()):,}")
```

VIII. Para reflexionar

- Comparen el número de parámetros de LeNet-5 (~62,000) con el MLP y la CNN de las prácticas anteriores. ¿Por qué LeNet-5 logra buen desempeño en MNIST con tan pocos parámetros?
- ¿Qué pasaría con el accuracy si se reemplaza ReLU por tanh (la activación original)? Pueden probarlo cambiando una línea del código.
- LeNet-5 usa padding=2 para simular la entrada de 32×32. ¿Qué pasaría si se elimina ese padding y se deja la imagen en 28×28 sin ajuste? (Pista: revisen las dimensiones resultantes en cada capa.)

IX. Referencia

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 86(11), 2278–2324.